Hands-on Kubernetes Troubleshooting

Build, Deploy, Break, Observe, Debug, Fix and Learn

For Developers and Infrastructure Teams

Single-node K3s

Each participant works on an isolated VM.

Real app

Flask API, PostgreSQL, migrations and volumes.

Troubleshooting-first

Every concept is attached to a failure mode.

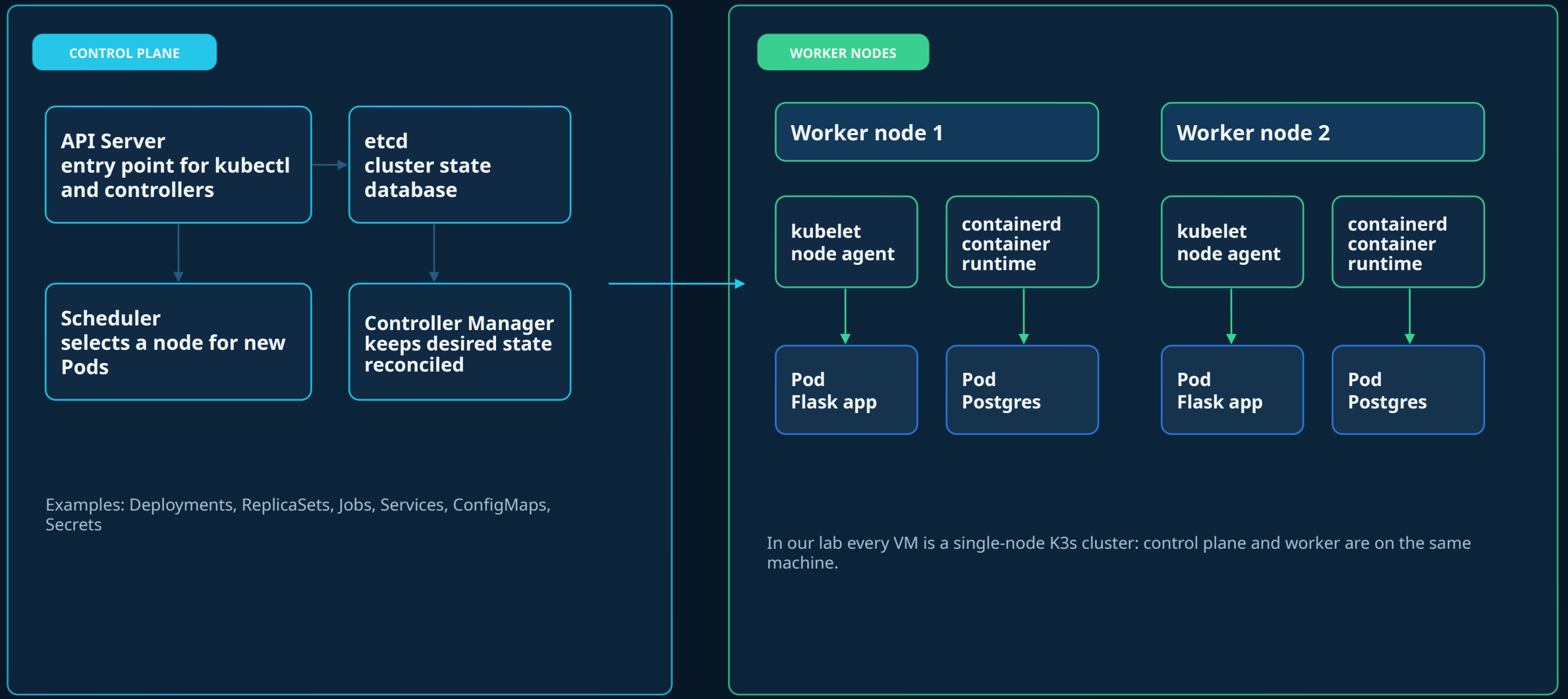
git clone https://github.com/slaneslane/k8s-workshop-2

Module 0

Basic facts

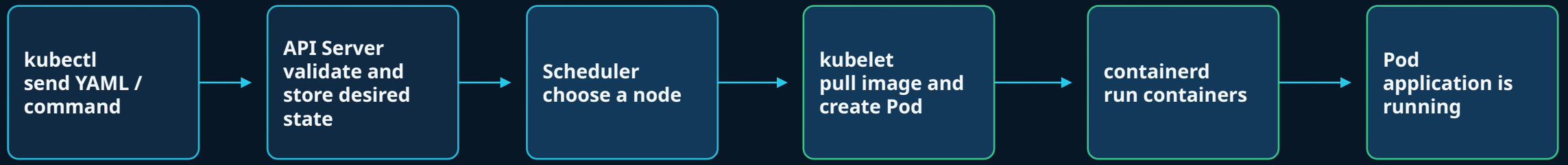
Kubernetes architecture: control plane and workers

The control plane stores desired state. Worker nodes run the actual application Pods.



What happens when you create a Pod?

A Deployment request becomes running containers through a series of control-plane and node actions.



Where common failures show up

ImagePullBackOff	kubelet/containerd cannot pull image	describe pod + events
CrashLoopBackOff	image was pulled, process exits	logs --previous + describe
Running but not Ready	process runs, readiness check fails	describe pod + endpoints
Pending	scheduler or admission cannot place/create Pod	describe pod + quota/events

Mental model: first classify the failure stage, then choose the command. Do not debug blindly.

VM vs Docker vs Kubernetes

Three different layers of abstraction, often used together.

Virtual Machine

Full OS boundary

- App
- Guest OS
- Virtual hardware
- Hypervisor / cloud

Best for strong isolation, legacy workloads, and OS-level control.

Docker

Single-container runtime

- Container image
- Container process
- Docker daemon
- Host OS kernel

Best for packaging and running a single container locally.

Kubernetes

Application platform

- Deployment / Service
- Pods
- Scheduler + controllers
- Nodes + container runtime

Best for operating many containers with desired state and self-healing.

In this workshop: VM provides the lab machine → Docker builds images → Kubernetes runs and operates the application.

YAML in Kubernetes — the minimum you need

Kubernetes manifests are structured text files that describe the desired state

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: flask-app
          image: k8s-workshop-app:debug
```

Indentation matters: use spaces, not tabs

key: value defines properties

Lists start with '-'

Nested blocks represent object structure

Kubernetes reads this as desired state

YAML is data, not a script

Reading a Kubernetes manifest

Most Kubernetes YAML files follow the same mental model

```yaml
apiVersion: v1          # Which API version?
kind: Service           # What object type?

metadata:
  name: flask-app       # Object name

spec:
  selector:
    app: flask-app      # Which Pods?
  ports:
    - port: 80          # Service port
      targetPort: 8080 # Container port
```

apiVersion + kind: what Kubernetes object this is

metadata: name, labels, annotations

spec: the desired configuration

status: current state, generated by Kubernetes

Troubleshooting often means comparing spec vs status

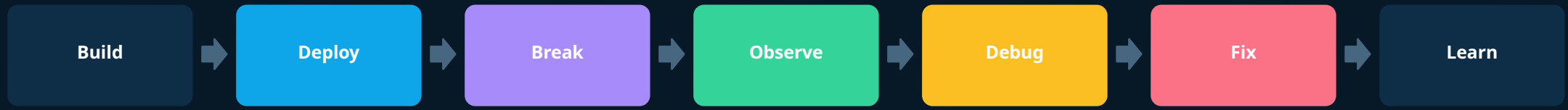
kubectl apply sends spec; Kubernetes updates status

Module 1

Workshop architecture and operating model

Workshop story

Every topic appears because the application needs it



Who is this for?

Same lab, different responsibilities

Developers

- Build images safely
- Understand runtime configuration
- Read logs and health endpoints
- Know when the app is the problem
- Design for debuggability

Infrastructure

- Understand workload impact
- Diagnose scheduling and storage
- Enforce quotas and defaults
- Support minimal images
- Provide reliable cluster signals

What we will build

One application, many failure modes

Application

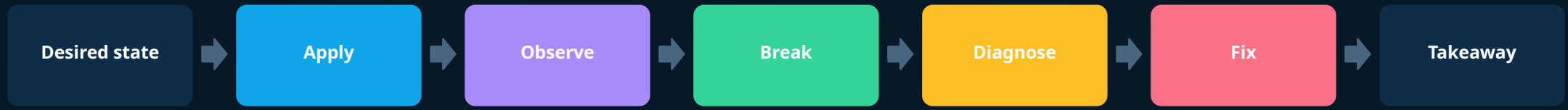
- Flask API
- PostgreSQL dependency
- Migration Job
- Health and readiness endpoints
- Cyclic logs and file logs

Kubernetes pieces

- Deployment and Service
- ConfigMap and Secret
- PVC and StatefulSet
- ResourceQuota and LimitRange
- Ephemeral debug containers

Lab rhythm

Repeat this pattern in every module



Single-node K3s assumption

Designing exercises for the real lab environment

We can demonstrate

- Pod lifecycle
- ImagePullBackOff
- CrashLoopBackOff
- Readiness failures
- emptyDir and RWO PVC
- Quota and OOMKilled

We discuss, not lab

- Multi-node RWO attach conflicts
- RWX storage implementation
- Production-grade PostgreSQL HA
- NetworkPolicy deep dive
- Full CI/CD pipeline

Recommended schedule

Timing and workshop flow

Day 1	**Build and first failures**	Images, Deployment, Service, ImagePullBackOff, CrashLoopBackOff, readiness.
Day 2	**Production-like application**	PostgreSQL, migration Job, volumes, distroless, resources, quotas, OOMKilled.
Day 3	**Playbook and combined cases**	Decision tree, team exercise, dev/platform responsibility split.

Module 2

VM setup and K3s installation

Recommended lab VM

One VM per participant

VM shape

- 4 vCPU
- 8 GB RAM
- 40 GB disk
- Ubuntu 24.04
- Single-node K3s

Why this shape

- Enough room for K3s + Docker
- PostgreSQL and several Pods
- Build cache without immediate disk pressure
- Room for OOMKilled lab
- Predictable participant experience

Install K3s

Basic single-node installation

```
curl -sfL https://get.k3s.io | sh -

sudo systemctl status k3s --no-pager
sudo kubectl get nodes


# For the workshop user:
sudo mkdir -p /home/k8s/.kube
sudo cp /etc/rancher/k3s/k3s.yaml /home/k8s/.kube/config
sudo chown -R k8s:k8s /home/k8s/.kube
```

Install K3s behind a proxy

Export proxy variables before running the installer

```
export HTTP_PROXY=http://proxy.example.com:3128
export HTTPS_PROXY=http://proxy.example.com:3128
export NO_PROXY=localhost,127.0.0.1,10.42.0.0/16,10.43.0.0/16,.svc,.cluster.local

curl -sfL https://get.k3s.io | sh -

sudo cat /etc/systemd/system/k3s.service.env
sudo systemctl restart k3s
```

Docker proxy

Docker daemon and Docker build need proxy separately

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf <<EOF
[Service]
Environment="HTTP_PROXY=http://proxy.example.com:3128"
Environment="HTTPS_PROXY=http://proxy.example.com:3128"
Environment="NO_PROXY=localhost,127.0.0.1,.svc,.cluster.local,10.42.0.0/16,10.43.0.0/16"
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker

docker info | grep -i proxy
```

Environment validation

Run before the workshop starts

Commands

- kubectl get nodes
- kubectl get pods -A
- docker ps
- docker compose version
- df -h
- free -h

Expected result

- Node is Ready
- K3s system Pods are Running
- Docker command works without sudo or after newgrp
- Enough disk space left
- Proxy pulls images successfully

Nginx quick start

A small hands-on warm-up before the Flask workshop application

Docker run | K3s namespace | Pod | Service | Port-forward

```
docker run -d --name nginx-demo -p 8080:80 nginx:latest

kubectl create namespace nginx-demo
kubectl -n nginx-demo run nginx --image=nginx --port=80
```

Goal: prove that participants can start a container locally and then run the same image in Kubernetes.

Run nginx with Docker

A minimal local container run: image, container, port mapping.

```
docker run -d --name nginx-demo -p 8080:80 nginx:latest
```

```
curl http://localhost:8080

docker ps
docker logs nginx-demo
docker rm -f nginx-demo
```

What this demonstrates

1. Image is pulled
2. Container starts
3. Host port maps to container port
4. HTTP responds on localhost

Local success before Kubernetes

Docker command anatomy

Keep the mental model simple before moving to Kubernetes.

```
docker run -d --name nginx-demo -p 8080:80 nginx:latest
```

-d	Run in the background
--name nginx-demo	Give the container a human-readable name
-p 8080:80	Host port 8080 → container port 80
nginx:latest	Image name and tag

Next: Kubernetes will run the same image, but the objects are different.

K8S – quick: Step 1 — Create a namespace

Use a small isolated namespace for the nginx warm-up.

```
kubectl create namespace nginx-demo

kubectl get namespaces

kubectl config set-context --current --namespace=nginx-demo
```

Cleanup later:

```
kubectl delete namespace nginx-demo
```

Why namespace first?

- keeps lab objects separate
- makes cleanup easy
- avoids touching the main workshop namespace

K8S - quick: Step 2 — Run nginx as a Pod

This is intentionally simple: one Pod, no Deployment yet.

```
kubectl -n nginx-demo run nginx --image=nginx:latest --port=80
```

```
kubectl -n nginx-demo get pods -w

kubectl -n nginx-demo describe pod nginx

kubectl -n nginx-demo logs nginx
```

What to observe

This is a smoke test of image pulling, scheduling, container start and basic logging.

K8S – quick: Step 3 — Expose the Pod with a Service

A Service gives the Pod a stable network entry point.

```
kubectl -n nginx-demo expose pod nginx --name=nginx --port=80 --target-port=80
```

```
kubectl -n nginx-demo get svc

kubectl -n nginx-demo get endpoints nginx

kubectl -n nginx-demo describe svc nginx
```

Without endpoints, Service has nowhere to send traffic.

K8S – quick: Step 4 — Port-forward and test

Forward traffic from your local machine to the Service.

```
kubectl -n nginx-demo port-forward svc/nginx 8080:80
```

```
curl http://localhost:8080

# or open in a browser:
# http://localhost:8080
```

Troubleshooting checks

- Is the Pod Running?
- Does the Service have endpoints?
- Is the targetPort correct?
- Is another process using local port 8080?

```
kubectl -n nginx-demo get pod,svc,endpoints
```

K8S – quick: From nginx to the workshop application

The warm-up introduces the mechanics; the Flask app introduces real dependencies and failure modes.

Next application adds:

- local registry and custom images
- ConfigMap and Secret
- PostgreSQL and migrations
- readiness depending on DB
- volumes, limits, quotas and real troubleshooting

```
kubectl delete namespace nginx-demo
```

Entering Containers (Docker vs Kubernetes)

```
Docker

docker ps
docker exec -it <container> sh
# or
 docker exec -it <container> bash

Example:
docker exec -it docker-app-1 sh

--------------------------------

Kubernetes

kubectl get pods
kubectl exec -it <pod> -- sh
# or
kubectl exec -it <pod> -- bash

Example:
kubectl -n k8s-workshop exec -it deploy/flask-app -- sh

--------------------------------
```

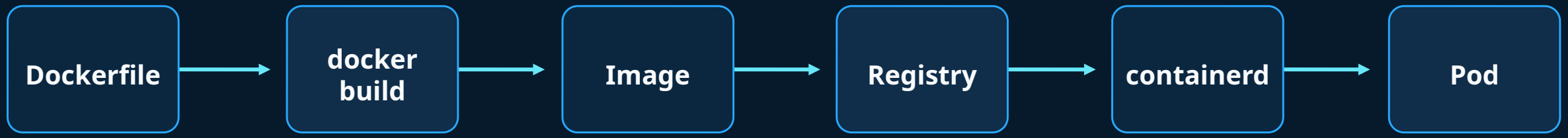
Module 2 Add-on

Local Container Registry

Build images locally, push to localhost:5000, then let K3s pull from the same VM.

Where does Kubernetes get images from?

Kubernetes pulls images from a registry; it does not automatically use your local Docker image cache.



- No registry means no pull path for Kubernetes
- Using localhost:5000 keeps the workshop fast and reproducible
- Later failures map directly to ImagePullBackOff

Registry options

For this workshop we use a local registry on every VM.

Public

- Docker Hub
- GHCR
- Quay
- Fast to start, but depends on Internet and rate limits

Private

- Harbor
- Artifactory
- ECR / ACR
- Production-like, but needs credentials and setup

Local

- localhost:5000
- No external dependency
- Great for labs
- Easy to break and fix

Start the local registry

The setup script runs a registry container and verifies push/pull before the workshop begins.

```
docker run -d \
  --name local-registry \
  --restart always \
  -p 5000:5000 \
  -v /opt/local-registry:/var/lib/registry \
  registry:2
```

Why local?

- No Docker Hub rate limits
- No shared credentials
- Predictable and fast
- Each participant owns their environment

workshop-friendly | offline-capable | repeatable

```
curl localhost:5000/v2/_catalog
```

Expected result: the registry API responds, even before any images are pushed.

Configure K3s to use the local registry

K3s uses containerd, so the Docker daemon configuration alone is not enough.

```
# /etc/rancher/k3s/registries.yaml
mirrors:
  "localhost:5000":
    endpoint:
      - "http://localhost:5000"
  "127.0.0.1:5000":
    endpoint:
      - "http://127.0.0.1:5000"

configs:
  "localhost:5000":
    tls:
      insecure_skip_verify: true
```

After changing it:

```
sudo systemctl restart k3s
kubectl get nodes
```

This reloads containerd registry configuration used by K3s.

Build workshop images with registry tags

Tag images with localhost:5000 from the beginning.

```
docker build -f docker/Dockerfile.debug -t localhost:5000/k8s-workshop-app:debug .

docker build -f docker/Dockerfile.distroless -t localhost:5000/k8s-workshop-app:distroless .

docker build -f docker/Dockerfile.migration -t localhost:5000/k8s-workshop-migration:debug .

docker build -f docker/Dockerfile.broken_migration -t localhost:5000/k8s-workshop-migration:broken .

docker build -f docker/Dockerfile.repair_migration -t localhost:5000/k8s-workshop-migration:repair .
```

Key point: k8s-workshop-app:debug is only a local Docker tag. localhost:5000/k8s-workshop-app:debug is a pullable registry reference.

Push and verify images

Pushing images makes them available for K3s/containerd to pull.

```
docker push localhost:5000/k8s-workshop-app:debug
docker push localhost:5000/k8s-workshop-app:distroless
docker push localhost:5000/k8s-workshop-migration:debug
docker push localhost:5000/k8s-workshop-migration:broken
docker push localhost:5000/k8s-workshop-migration:repair
```

```
curl localhost:5000/v2/_catalog
curl localhost:5000/v2/k8s-workshop-app/tags/list
curl localhost:5000/v2/k8s-workshop-migration/tags/list
```

Use registry references in Kubernetes manifests

The Deployment should point to the local registry image.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  template:
    spec:
      containers:
        - name: flask-app
          image: localhost:5000/k8s-workshop-app:debug
          imagePullPolicy: IfNotPresent
```

Recommended for labs

- imagePullPolicy: IfNotPresent
- fast local pulls
- change tag when you want a new image
- use Always when teaching pull behavior

Troubleshooting preview: ImagePullBackOff

A registry flow makes image problems visible and easy to diagnose.

```
# typo in tag
image: localhost:5000/k8s-workshop-app:debgu

# expected status
ImagePullBackOff
```

First checks

```
kubectl describe pod <pod>
kubectl get events --sort-by=.lastTimestamp
curl localhost:5000/v2/_catalog
```

Mental model

Lab: Build, push, deploy

This lab becomes the foundation for the rest of the workshop.

Checklist

- Run setup-local-registry.sh and confirm the test passes
- Build app, distroless, and migration images with localhost:5000 tags
- Push all images to the local registry
- Verify registry catalog and image tags
- Use localhost:5000/... image references in Kubernetes manifests
- Deploy the app in Module 3

Build | Push | Pull | Deploy | Troubleshoot

Module 3

Containers, images and local application

Container image mental model

Image is not a VM

Image contains

- Application code
- Runtime dependencies
- Entrypoint and command
- Default environment
- Filesystem layers

Image does not contain by default

- Persistent data
- Cluster configuration
- Kubernetes Service identity
- Secrets from the cluster
- A guarantee that shell exists

Image build flow

Multi-stage builds separate build-time and runtime concerns

Debug image vs distroless image

Both are useful, for different reasons

Debug-friendly image

- Based on python:slim
- Contains shell and tools
- Easier kubectl exec
- Larger attack surface
- Useful for labs and diagnosis

Distroless image

- No shell
- Fewer packages
- Smaller attack surface
- Requires better logs/probes
- Use kubectl debug when needed

Build images

Commands used in the repository

```
docker build -f docker/Dockerfile.debug     -t localhost:5000/k8s-workshop-app:debug .

docker build -f docker/Dockerfile.distroless -t localhost:5000/k8s-workshop-app:distroless .

docker build -f docker/Dockerfile.migration -t localhost:5000/k8s-workshop-migration:debug .

docker build -f docker/Dockerfile.broken_migration -t localhost:5000/k8s-workshop-migration:broken .

docker build -f docker/Dockerfile.repair_migration -t localhost:5000/k8s-workshop-migration:repair .


docker image ls | grep k8s-workshop
```

Run locally with Docker Compose

Prove the app before Kubernetes

```
cd docker
docker compose up --build

curl http://localhost:8080/items

curl http://localhost:8080/healthz

curl http://localhost:8080/readyz

curl http://localhost:8080/version

curl -X POST http://localhost:8080/items -H 'Content-Type: application/json' -d '{"name":"demo"}'
```

Application endpoints

The app is designed to create useful signals

Operational endpoints

- /healthz - process is alive
- /readyz - DB dependency is available
- /version - image/config visible
- /file-log - last file log lines

Training endpoints

- /items - DB-backed data path
- /stress/cpu - CPU behavior
- /stress/memory - OOMKilled lab
- cyclic log writer - restart visibility

Module 4

First deployment to Kubernetes

Deployment, Pod and Service

Three objects, three jobs

Deployment

- Declarative desired state
- Creates ReplicaSets
- Recreates failed Pods
- Supports rollout and rollback
- Best default for stateless app

Service

- Stable virtual IP and DNS
- Selects Pods by labels
- Routes to ready endpoints
- Decouples clients from Pods
- Debug with endpoints

First deploy – dependencies and DB migration

Apply manifests in order

```
kubectl apply -f k8s/base/01-namespace.yaml

kubectl apply -n k8s-workshop -f k8s/base/02-configmap.yaml

kubectl apply -n k8s-workshop -f k8s/base/03-secret.yaml

kubectl apply -n k8s-workshop -f k8s/base/04-postgres-statefulset.yaml

kubectl -n k8s-workshop rollout status statefulset/postgres --timeout=180s

kubectl delete job db-migration -n k8s-workshop --ignore-not-found=true

kubectl apply -n k8s-workshop -f k8s/base/05-migration-job.yaml

kubectl -n k8s-workshop wait --for=condition=complete job/db-migration --timeout=120s

kubectl -n k8s-workshop logs job/db-migration

kubectl -n k8s-workshop get deploy,rs,pod,svc,endpoints
```

First deploy – application and service

Apply manifests in order

```
kubectl apply -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl apply -n k8s-workshop -f k8s/base/07-app-service.yaml

kubectl -n k8s-workshop rollout status deployment/flask-app --timeout=120s

# Verify

kubectl -n k8s-workshop get deploy,rs,pod,svc,endpoints,jobs,pvc

kubectl -n k8s-workshop port-forward svc/flask-app 8080:80


# And in the second terminal:

curl http://localhost:8080/healthz

curl http://localhost:8080/readyz

curl http://localhost:8080/version
```

Observe first

Do not fix before observing

```
kubectl -n k8s-workshop get pods -o wide

kubectl -n k8s-workshop describe pod <pod-name>

kubectl -n k8s-workshop logs <pod-name>

kubectl -n k8s-workshop get events --sort-by=.lastTimestamp

kubectl -n k8s-workshop port-forward svc/flask-app 8080:80
```

Rollout model

Every rollout is a controlled state transition

Rollout commands

Useful in almost every lab

```
kubectl -n k8s-workshop rollout status deploy/flask-app

kubectl -n k8s-workshop rollout history deploy/flask-app

kubectl -n k8s-workshop rollout restart deploy/flask-app

kubectl -n k8s-workshop rollout undo deploy/flask-app

kubectl -n k8s-workshop set image deploy/flask-app flask-app=localhost:5000/k8s-workshop-app:debug
```

Module 5

Fast failure diagnosis

First classification

Classify the state before reading random logs

State	Likely phase	First command	Typical root cause
Pending	Scheduling/admission	kubectl describe pod	quota, capacity, PVC, node selector
ImagePullBackOff	Image pull	kubectl describe pod	bad tag, registry auth, proxy
CrashLoopBackOff	Runtime crash	kubectl logs --previous	bad command, config, app error
Running not Ready	Health gate	describe + logs	readiness path, dependency down
OOMKilled	Runtime limit	describe pod	memory limit exceeded
No endpoints	Service routing	kubectl get endpoints	selector mismatch or not ready

Fast Failure Diagnosis

Use module-local manifests, not old challenges/solutions paths

- Start from a healthy Module 4 baseline
- Apply one faulty manifest from modules/module-05-fast-failure-diagnosis/
- Observe the symptom before fixing anything
- Recover from the reference manifest in k8s/base/

```
cd ~/k8s-workshop-2
ls modules/module-05-fast-failure-diagnosis

# Working reference:
k8s/base/06-app-deployment.yaml
```

Observe → Describe → Logs/Events → Recover

ImagePullBackOff

The container never started

What happened

- Kubelet tried to pull the image
- Pull failed
- Retry delay increases
- There are no app logs yet
- Events contain the clue

Common causes

- Typo in image name
- Wrong tag
- Private registry without secret
- Proxy/certificate/DNS issue
- ImagePullPolicy surprises

Challenge 1: ImagePullBackOff

Break and diagnose

```
kubectl delete -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl apply -n k8s-workshop -f modules/module-05-fast-failure-diagnosis/01-imagepullbackoff.yaml

kubectl -n k8s-workshop get pods

kubectl -n k8s-workshop describe pod <pod-name>

kubectl -n k8s-workshop get events --sort-by=.lastTimestamp
```

Expected: ErrImagePull → ImagePullBackOff

There are no application logs yet

The clue is in Pod events and image name/tag

Question: did the process ever start?

Quick fix — kubectl edit

Use only for quick investigation or emergency edits — keep Git/YAML as the source of truth

```
# ImagePullBackOff example
kubectl -n k8s-workshop edit deploy/flask-app

# Find the container image:
image: localhost:5000/k8s-workshop-app:debgu

# Fix the tag:
image: localhost:5000/k8s-workshop-app:debug

# Save and exit editor
```

- Good for showing live object editing
- Creates a new Deployment revision
- Then watch the rollout status
- Do not use as the normal workshop

```
kubectl -n k8s-workshop rollout status deploy/flask-app
kubectl -n k8s-workshop get pods
```

After the demo, restore from k8s/base/06-app-deployment.yaml to keep the lab deterministic

Recovery reveal — ImagePullBackOff

Show after participants identify the bad image tag

```
kubectl delete -n k8s-workshop \
-f modules/module-05-fast-failure-diagnosis/01-imagepullbackoff.yaml

kubectl apply -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl -n k8s-workshop rollout status deployment/flask-app --timeout=120s

kubectl -n k8s-workshop get pods
```

We do not keep a separate solution YAML

The working source of truth is k8s/base/

This also avoids old challenges/solutions paths

CrashLoopBackOff

The image starts, then the process exits

What happened

- Image was pulled
- Container started
- Main process exited
- Kubernetes restarted it
- Backoff delay increased

Use these signals

- kubectl logs <pod>
- kubectl logs <pod> --previous
- exit code in describe
- environment variables
- command and args

Challenge 2: CrashLoopBackOff

The key command is logs --previous

```
kubectl delete -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl apply -n k8s-workshop -f modules/module-05-fast-failure-diagnosis/02-crashloopbackoff.yaml

kubectl -n k8s-workshop get pods -w

kubectl -n k8s-workshop logs <pod-name>

kubectl -n k8s-workshop logs <pod-name> --previous

kubectl -n k8s-workshop describe pod <pod-name>
```

Expected: CrashLoopBackOff

Use --previous for the last failed container

Check command, args, env and exit code

Key command: logs --previous

Recovery reveal — CrashLoopBackOff

Restore the normal Deployment template

```
kubectl delete -n k8s-workshop \
-f modules/module-05-fast-failure-diagnosis/02-crashloopbackoff.yaml

kubectl apply -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl -n k8s-workshop rollout status deployment/flask-app --timeout=120s

kubectl -n k8s-workshop logs deployment/flask-app
```

The faulty manifest overrides command/args

The base Deployment restores the real app startup

Verify the rollout, then move on

Running but not Ready

Running is not the same as receiving traffic

Readiness means

- The process is running
- Kubernetes asked the app if it is ready
- The app said no or did not respond
- Service endpoints exclude the Pod
- Traffic should not be routed

Common causes

- Wrong readiness path
- Wrong port or targetPort
- Dependency unavailable
- DB credentials wrong
- App takes longer to start

Challenge 3: Readiness failure

Pod runs, Service has no ready endpoint

```
kubectl delete -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl apply -n k8s-workshop -f modules/module-05-fast-failure-diagnosis/03-readiness-failure.yaml

kubectl -n k8s-workshop get pods

kubectl -n k8s-workshop describe pod <pod-name>

kubectl -n k8s-workshop get endpoints

kubectl -n k8s-workshop logs <pod-name>
```

Expected: Running, but 0/1 Ready

Readiness probe uses the wrong path

Service endpoints exclude unready Pods

Running ≠ Ready

Recovery reveal — Readiness failure

Restore readiness probe from the base manifest

```
kubectl delete -n k8s-workshop \
-f modules/module-05-fast-failure-diagnosis/03-readiness-failure.yaml

kubectl apply -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl -n k8s-workshop rollout status deployment/flask-app --timeout=120s

kubectl -n k8s-workshop get endpoints flask-app
```

- The app was running, but excluded from traffic
- Fix readiness before testing Service routing
- Use endpoints as a quick Service readiness check

Module 6

PostgreSQL and database migrations

Adding PostgreSQL

Make the app more realistic

Kubernetes objects

- StatefulSet
- Headless or normal Service
- Secret for credentials
- PVC via volumeClaimTemplates
- Readiness depends on DB

Operational questions

- Where is data stored?
- When is DB ready?
- How do we migrate schema?
- What fails if Secret is wrong?
- What should restart automatically?

Recovery rules

Keep the flow short and deterministic

- Never delete PostgreSQL during this module
- Delete a Job before re-running the same Job name
- Use k8s/base/ to restore the normal app configuration
- Use repair Job to clean the controlled broken SQL state

```
# Cleanup after the module
kubectl delete job \
  db-migration \
  db-migration-broken \
  db-migration-repair \
  -n k8s-workshop \
  --ignore-not-found=true
```

Migrations should be atomic, idempotent, and observable

Deploy PostgreSQL and migration Job

The DB becomes part of the application lifecycle

```
# THIS IS ALREADY DONE !

kubectl apply -n k8s-workshop -f k8s/base/04-postgres-statefulset.yaml

kubectl -n k8s-workshop rollout status statefulset/postgres

kubectl -n k8s-workshop get pvc



kubectl apply -n k8s-workshop -f k8s/base/05-migration-job.yaml

kubectl -n k8s-workshop logs job/db-migration

kubectl -n k8s-workshop get jobs
```

Migration Job vs initContainer

Use the right mechanism for the risk

Migration Job

- Runs once per release step
- Easy to inspect logs
- Can fail before app rollout
- Avoids N replicas running migration
- Good teaching default

initContainer

- Runs before each Pod
- Good for local setup/waiting
- Can duplicate migration work
- Tightly coupled with Pod startup
- Useful but risky for schema changes

Challenge 4: DB connection failure

Wrong Secret or wrong Service DNS name

```
kubectl delete job db-migration db-migration-broken db-migration-repair -n k8s-workshop \
--ignore-not-found=true


kubectl apply -n k8s-workshop -f modules/module-06-postgresql-and-database-migrations/01-wrong-database-secret.yaml

kubectl -n k8s-workshop rollout restart deploy/flask-app

kubectl -n k8s-workshop get pods

kubectl -n k8s-workshop logs deploy/flask-app

kubectl -n k8s-workshop describe pod <pod-name>


# Fix
kubectl apply -n k8s-workshop -f k8s/base/03-secret.yaml

kubectl apply -n k8s-workshop -f k8s/base/06-app-deployment,yaml

kubectl apply -n k8s-workshop rollout status deploy/flask-app
```

Challenge 5: Broken migration Job

Completed vs Failed matters

```
kubectl delete job db-migration db-migration-broken db-migration-repair -n k8s-workshop \
--ignore-not-found=true

kubectl apply -n k8s-workshop -f modules/module-06-postgresql-and-database-migrations/02-broken-migration-job.yaml

kubectl -n k8s-workshop get jobs,pods

kubectl -n k8s-workshop logs job/db-migration-broken

kubectl -n k8s-workshop describe job db-migration-broken


# Fix with a valid migration or fixed Job

kubectl apply -n k8s-workshop -f k8s/base/03-secret.yaml

kubectl apply -n k8s-workshop -f k8s/base/06-app-deployment.yaml

kubectl -n k8s-workshop rollout status deploy/flask-app
```

Image: migration:broken

Isolated schema: workshop_training

Expected: Job Failed

Key idea: partial DB state can remain

Challenge 5: Broken migration Job – cleanup

Completed vs Failed matters

```
kubectl delete job db-migration-repair -n k8s-workshop --ignore-not-found=true

kubectl apply -n k8s-workshop -f modules/module-06-postgresql-and-database-migrations/03-repair-migration-job.yaml

kubectl -n k8s-workshop wait --for=condition=complete job/db-migration-repair --timeout=120s

kubectl -n k8s-workshop logs job/db-migration-repair



kubectl -n k8s-workshop exec statefulset/postgres -- \

  psql -U workshop -d workshop -c \

  "SELECT table_schema, table_name

   FROM information_schema.tables

   WHERE table_schema = 'workshop_training';"
```

Module 7

Volumes and persistence

Volume types in this workshop

What we can and cannot demonstrate on single-node K3s

Type	Access	Survives Pod recreation?	Workshop use
emptyDir	Pod-local	No	Temporary file log
ConfigMap volume	Read-only files	N/A	Application configuration as files
Secret volume	Read-only files	N/A	Credential files
PVC RWO	Single node write	Yes	Persistent app log and PostgreSQL
PVC RWX	Many writers	Yes	Slides only: NFS/CephFS/EFS/Azure Files
PVC RWOP	Single Pod write	Yes	Conceptual comparison

emptyDir

Good for temporary data, not persistence

Useful for

- Scratch files
- Temporary caches
- Sidecar sharing inside a Pod
- Training log disappearance
- No storage provisioning needed

Not useful for

- Databases
- Data that must survive Pod recreation
- Cross-Pod sharing
- Node failure protection
- Shared state

emptyDir volume

Where did my file go?

```
# Create a Pod with emptyDir storage

kubectl apply -n k8s-workshop -f k8s/storage/01-emptydir-app.yaml

# Observe files being written

kubectl -n k8s-workshop exec flask-app-emptydir -- tail /data/events.log

# Delete the Pod

kubectl -n k8s-workshop delete pod flask-app-emptydir

# Recreate the Pod

kubectl apply -n k8s-workshop -f k8s/storage/01-emptydir-app.yaml

# Inspect the directory

kubectl -n k8s-workshop exec flask-app-emptydir -- ls -la /data
```

PVC with RWO

Persistent volume survives Pod recreation

What RWO means

- ReadWriteOnce
- Mounted read-write by a single node
- In K3s local-path creates local PVs
- Good for single-node persistence lab
- Not the same as shared storage

Single-node caveat

- Two Pods on the same node may appear to work
- Multi-attach errors need multi-node clusters
- Do not overclaim what the lab proves
- Teach semantics on slides
- Use StatefulSet for DB pattern

PVC RWO

File survives Pod recreation

```
kubectl apply -n k8s-workshop -f k8s/storage/02-pvc-rwo.yaml

kubectl -n k8s-workshop rollout status deployment/flask-app-pvc --timeout=120s

kubectl -n k8s-workshop port-forward deployment/flask-app-pvc 8082:8080
```

curl http://localhost:8082/file-log

```
kubectl -n k8s-workshop delete pod -l app=flask-app-pvc

kubectl -n k8s-workshop rollout status deployment/flask-app-pvc –timeout=120s

kubectl -n k8s-workshop port-forward deployment/flask-app-pvc 8082:8080
```

curl http://localhost:8082/file-log

```
kubectl -n k8s-workshop get pvc,pv
```

Storage lifecycle

Follow the data, not only the Pod

ConfigMap and Secret volumes

Configuration can also be mounted as files

ConfigMap volume

- Plain configuration
- Mounted as files
- Can update files eventually
- Not for sensitive data
- Good for application properties

Secret volume

- Sensitive values
- Mounted read-only
- Base64 in API object
- Still needs RBAC discipline
- Restart often needed for env vars

ConfigMap and Secret volume lab

Read configuration as files

```
kubectl apply -n k8s-workshop -f k8s/storage/03-configmap-secret-volumes.yaml

kubectl -n k8s-workshop get pod volume-reader

kubectl -n k8s-workshop exec volume-reader -- ls -la /config

kubectl -n k8s-workshop exec volume-reader -- ls -la /secret

kubectl -n k8s-workshop exec volume-reader -- cat /config/application.properties

kubectl -n k8s-workshop exec volume-reader -- cat /secret/password.txt
```

RWX overview

Discuss it, do not force it in single-node K3s

RWX means

- ReadWriteMany
- Multiple writers/readers
- Requires backend support
- Common for shared content
- Not every StorageClass supports it

Examples

- NFS
- CephFS
- Amazon EFS
- Azure Files
- NetApp / enterprise NAS

StatefulSet storage pattern

Databases need stable identity and storage

StatefulSet gives

- Stable Pod name
- Ordered rollout by default
- Stable network identity
- volumeClaimTemplates
- One PVC per replica

Teaching message

- Not HA PostgreSQL
- Good for storage lifecycle
- PVC remains after Pod recreation
- Deleting StatefulSet does not automatically delete PVCs
- Know what owns the data

Module 8

Distroless and no-shell troubleshooting

Why no shell?

Minimal images improve security but change troubleshooting

Benefits

- Smaller runtime surface
- Fewer vulnerable packages
- Less accidental tooling
- Closer to immutable runtime
- Good production default

Trade-offs

- kubectl exec -- sh fails
- No curl, ps, nslookup
- Logs and probes become critical
- Need debug containers
- Need good application telemetry

Distroless

Do not assume shell exists

```
kubectl apply -n k8s-workshop -f modules/module-08-distroless-and-no-shell-troubleshooting/01-distroless-deployment.yaml

kubectl -n k8s-workshop rollout status deployment/flask-app --timeout=120s

kubectl -n k8s-workshop get pods

kubectl -n k8s-workshop exec -it deployment/flask-app – sh


# Expected - exec: "sh": executable file not found
```

Distroless - debugging

Do not assume shell exists

```
kubectl -n k8s-workshop logs deployment/flask-app

kubectl -n k8s-workshop describe deployment/flask-app

kubectl -n k8s-workshop get events --sort-by=.lastTimestamp

kubectl -n k8s-workshop get endpoints flask-app


# but we can do this as well:

POD=$(kubectl -n k8s-workshop get pod -l app=flask-app  -o jsonpath='{.items[0].metadata.name}')

kubectl -n k8s-workshop debug -it pod/${POD} --image=busybox:1.36 --target=flask-app – sh


# cleanup after the lab:

kubectl apply -n k8s-workshop -f k8s/base/06-app-deployment.yaml
```

Debugging without shell

Use cluster signals before interactive access

Ephemeral debug containers

A safer operational escape hatch

Use when

- Target image has no shell
- Need network tools
- Need process namespace insight
- Need quick triage
- Do not want to rebuild app image

Remember

- Not a replacement for logs
- May need RBAC permission
- Does not change app container
- Still uses same Pod network namespace
- Use carefully in production

Module 9

Requests, limits and runtime behavior

Requests vs actual usage

This is the source of many misunderstandings

Requests

- Used by scheduler
- Counted by quota
- Reserve scheduling capacity
- Do not mean process is using it now
- Help cluster planning

Actual usage

- What the process consumes now
- Can be lower than request
- Can be higher than request if allowed
- Observed by metrics
- Drives performance and cost

Limits

Runtime guardrails are different from scheduling

CPU limit

- CPU is throttled
- Application becomes slower
- Usually not killed
- Can create latency
- Be careful with tight CPU limits

Memory limit

- Memory cannot be throttled the same way
- Exceeding limit can kill container
- Seen as OOMKilled
- Restart can cause CrashLoopBackOff
- Needs app-level understanding

QoS classes

How Kubernetes classifies Pods from resources

QoS	Condition	Operational meaning	Example
Guaranteed	requests == limits for all containers	Highest eviction priority protection	Critical stable workload
Burstable	some requests or limits set	Common default for apps	Most workshop app Pods
BestEffort	no requests or limits	First to be evicted under pressure	Avoid for important apps

OOMKilled

Memory limit becomes visible at runtime

```
kubectl apply -n k8s-workshop -f modules/module-09-requests-limits-and-runtime-behavior/01-low-memory-oom.yaml

kubectl -n k8s-workshop rollout status deployment/flask-app-oom --timeout=120s

kubectl -n k8s-workshop port-forward deployment/flask-app-oom 8081:8080

# in another terminal:
curl "http://localhost:8081/stress/memory?mib=256"

kubectl -n k8s-workshop get pods -w

kubectl -n k8s-workshop describe pod -l app=flask-app-oom

kubectl -n k8s-workshop logs -l app=flask-app-oom –previous

# Expected - Reason: OOMKilled - Exit Code: 137

# cleanup:

kubectl delete -n k8s-workshop -f modules/module-09-requests-limits-and-runtime-behavior/01-low-memory-oom.yaml
```

How to explain VM comparison

Useful for infra teams

Kubernetes Pod

- Requests guide scheduling
- Limits control runtime
- Actual usage can be much lower
- Quota may block creation
- Pod can be restarted/rescheduled

VM workload

- Flavor/guest memory shapes scheduling/capacity
- Memory pressure behaves differently
- Platform may overcommit CPU
- Quota/capacity can block creation
- Treat as heavier scheduling unit

Module 10

ResourceQuota and LimitRange

ResourceQuota

Namespace-level capacity control

What it controls

- Aggregate requests and limits
- Pod count
- PVC count and storage
- Object counts
- Admission decisions

What it teaches

- Why new Pods may not appear
- Why idle-looking clusters can reject work
- Why requests are part of capacity planning
- How teams share clusters
- Dev/platform contract

LimitRange

Defaults and guardrails for a namespace

Can provide

- Default requests
- Default limits
- Minimum allowed request
- Maximum allowed limit
- PVC size constraints

Why it matters

- Prevents BestEffort by accident
- Gives predictable scheduling
- Protects neighbors
- Can surprise developers
- Must be documented

Apply ResourceQuota and LimitRange

Use after the app basics are understood

```
kubectl apply -n k8s-workshop -f k8s/resources/01-limitrange.yaml

kubectl apply -n k8s-workshop -f k8s/resources/02-resourcequota.yaml



kubectl -n k8s-workshop describe quota

kubectl -n k8s-workshop describe limitrange
```

Challenge 8: Quota exceeded

Pods can fail before the app has a chance

```
kubectl apply -n k8s-workshop -f challenges/challenge-08-quota/broken.yaml

kubectl -n k8s-workshop get deploy,rs,pod

kubectl -n k8s-workshop describe quota

kubectl -n k8s-workshop describe pod <pending-pod>

kubectl -n k8s-workshop get events --sort-by=.lastTimestamp
```

Key message for participants

Do not confuse booked, allowed and used

Booked / requested

- Used for scheduling
- Counted by quota
- Can block new Pods
- Represents expected need
- Not necessarily used now

Used / runtime

- Measured by metrics
- Changes over time
- Can exceed request
- Limited by limits
- Can cause throttling or OOM

Module 11

Service routing and application availability

Service is label-based

The Service does not know your Deployment name

Service selects

- Pods by labels
- Only ready endpoints
- targetPort on Pod
- Stable DNS name
- Virtual IP inside cluster

Common failures

- Selector mismatch
- Wrong targetPort
- Pod not Ready
- Namespace mismatch
- Network/proxy confusion

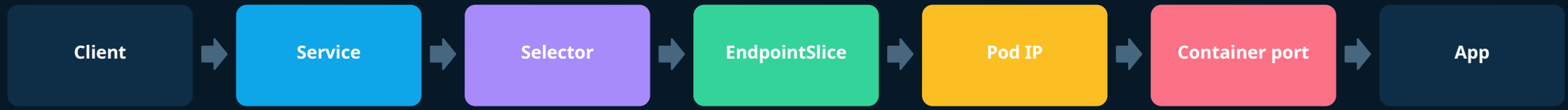
Challenge 10: Wrong Service selector

Application runs, but no traffic reaches it

```
kubectl apply -n k8s-workshop -f challenges/challenge-10-service-selector/broken.yaml

kubectl -n k8s-workshop get pods --show-labels

kubectl -n k8s-workshop describe svc selector-app

kubectl -n k8s-workshop get endpoints selector-app


# Fix selector
kubectl apply -n k8s-workshop -f challenges/challenge-10-service-selector/solution.yaml
```

Request path

Where to look when traffic fails



Module 12

Troubleshooting playbook

Decision tree

Use this as the final participant cheat sheet

Question	Yes means	First command	Next step
Pod not created?	Controller/admission issue	describe deploy/rs	events, quota
Pod Pending?	Cannot schedule/start	describe pod	capacity, PVC, quota
ImagePullBackOff?	Image not pulled	describe pod	tag, registry, proxy
CrashLoopBackOff?	Process exits	logs --previous	command, env, app
Running not Ready?	Health gate failed	describe + logs	probe/dependency
OOMKilled?	Memory limit hit	describe pod	limits/usage
No endpoints?	Service cannot route	get endpoints	selector/readiness

Core command set

Commands participants should leave with

```
kubectl get pods -o wide

kubectl describe pod <pod>

kubectl logs <pod>

kubectl logs <pod> --previous

kubectl get events --sort-by=.lastTimestamp

kubectl get deploy,rs,pod,svc,endpoints,pvc

kubectl describe quota

kubectl rollout status deploy/<name>

kubectl debug -it pod/<pod> --image=busybox:1.36 --target=<container> -- sh
```

How to run combined cases

Final exercise format

Instructor setup

- Apply one broken manifest
- Do not reveal the failure mode
- Ask team to classify state
- Ask for first three commands
- Ask for root cause and fix

Participant output

- Observed state
- Evidence from events/logs
- Root cause
- Fix command or YAML change
- Preventive practice

Final 2-hour agenda

Timing and workshop flow

30 min	**End-to-end recap**	Build -> Deploy -> DB -> Migrate -> Persist -> Limit.
30 min	**Troubleshooting decision tree**	Classify states and map each to first commands.
40 min	**Combined scenarios**	Teams diagnose three broken manifests.
20 min	**Dev vs Platform contract**	Who defines what and what should be standardized.

Dev vs platform contract

Make responsibilities explicit

Developers should define

- Health and readiness behavior
- Resource expectations
- Configuration keys
- Migration strategy
- Meaningful logs

Platform should define

- Default requests/limits
- Quota and namespace policy
- StorageClass options
- Registry and proxy access
- Debug permissions and guardrails

Production checklist

What to take back to real work

Application

- Logs to stdout/stderr
- Readiness reflects dependencies
- No secrets in images
- Migration plan is explicit
- Resource requests are tested

Platform

- Namespace defaults documented
- Quota visible to teams
- Storage semantics clear
- Image registry access reliable
- Troubleshooting access controlled

Closing message

Kubernetes is observable when you ask the right questions